

Implementing Content-Based Filtering in TensorFlow

This video walks through the specific code structure for building the "Two-Tower" neural network architecture using the TensorFlow Keras API.

1. Defining the Networks

We create two separate Sequential models: one for the user and one for the item (movie). Both must end with an output layer of the same size (e.g., 32 units).

The User Network:

```
1 user_NN = tf.keras.models.Sequential([
2     # Hidden layers with Relu activation
3     tf.keras.layers.Dense(256, activation='relu'),
4     tf.keras.layers.Dense(128, activation='relu'),
5     # Output layer (the "User Vector")
6     tf.keras.layers.Dense(32)
7 ])
```

The Item (Movie) Network:

```
1 item_NN = tf.keras.models.Sequential([
2     # Hidden layers with Relu activation
3     tf.keras.layers.Dense(256, activation='relu'),
4     tf.keras.layers.Dense(128, activation='relu'),
5     # Output layer (the "Item Vector")
6     tf.keras.layers.Dense(32)
7 ])
```

2. The Forward Pass and Normalization

We define the input layers, pass them through the networks to get the vectors (v_u and v_m), and then apply a crucial normalization step.

Why Normalize?

Normalizing the vectors to have a length of 1 (L2 norm) significantly improves the algorithm's performance.

```
1 # 1. Create Inputs
2 input_user = tf.keras.layers.Input(shape=(num_user_features))
3 input_item = tf.keras.layers.Input(shape=(num_item_features))
4
5 # 2. Compute Vectors (Pass inputs through the networks)
6 vu = user_NN(input_user)
7 vm = item_NN(input_item)
8
9 # 3. L2 Normalization (Scale vectors to length 1)
10 vu = tf.linalg.l2_normalize(vu, axis=1)
11 vm = tf.linalg.l2_normalize(vm, axis=1)
```

3. The Dot Product and Model Definition

Finally, we calculate the predicted rating by taking the dot product of the two normalized vectors.

```
1 # 4. Compute Dot Product (The Prediction)
2 # Keras has a specific layer for this operation
3 output = tf.keras.layers.Dot(axes=1)([vu, vm])
4
5 # 5. Define the Full Model
6 # Inputs: User features AND Item features
7 # Output: The dot product prediction
8 model = tf.keras.Model([input_user, input_item], output)
9
10 # 6. Compile the Model
11 # Use Mean Squared Error cost function
12 model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.01),
13               loss='mean_squared_error')
```

Summary of Course 3, Week 2

You have now completed the deep dive into Recommender Systems!

- **Collaborative Filtering:** Recommendations based on user/item interactions (ratings).
- **Content-Based Filtering:** Recommendations based on user/item features using Neural Networks.
- **Architecture:** Retrieval (fast candidate generation) → Ranking (precise scoring).
- **Ethics:** The importance of optimizing for user well-being, not just profit or engagement.